

E-PAY Setup Guide

Getting the E-Banking System up and running on your machine

Before You Begin

Before diving into the setup, make sure your machine has the following tools installed. If you're missing any of them, follow the links below to download and install them — the setup won't work without these.

- Python 3.11 or newer — the backend runs entirely on Python
- Node.js 18 or newer — needed to build and serve the frontend
- npm — this comes bundled with Node.js, so you'll get it automatically

Step 1 — Get the Code

Start by cloning the repository to your local machine and navigating into the project folder. Open your terminal and run:

```
git clone <repo-url> E_PAY
cd E_PAY/e_banking
```

Step 2 — Set Up the Backend

Navigate into the backend folder and install the required Python packages. These libraries power everything from the API server to encryption and database connectivity.

```
cd backend
pip install -r requirements.txt
cd ..
```

This will install: Flask, flask-cors, pycryptodome, supabase, python-dotenv, gunicorn, and psycopg2-binary.

Step 3 — Set Up the Frontend

Now install the frontend dependencies. From the e_banking folder, run:

```
cd frontend
```

```
npm install
cd ..
```

Step 4 — Connect to Supabase

E-PAY uses Supabase as its cloud database. You'll need to point the app at a Supabase project by filling in your credentials. Open the file at `backend/.env.backend` and update these three values:

```
SUPABASE_URL=https://<your-project>.supabase.co
SUPABASE_KEY=<your-anon-key>
SUPABASE_SERVICE_ROLE_KEY=<your-service-role-key>
```

Note: *The repository may already include a pre-configured project that works out of the box. If it doesn't, create a free account at supabase.com, start a new project, and paste your URL and keys here.*

Setting Up the Database Schema

Once your credentials are in place, you need to apply the database schema. Pick whichever option suits you:

Option A — Using the Supabase Dashboard (easiest):

- Log in at supabase.com and open your project
- Go to the SQL Editor
- Paste the contents of `database/SUPABASE_NEW_DATABASE_SETUP.sql` and click Run

Option B — Using the command line:

```
cd backend
python init_database.py
```

Note: *This option requires `SUPABASE_DB_PASSWORD` to be set in your `.env.backend` file.*

Step 5 — Run the Application

There are two ways to run E-PAY depending on whether you want a quick production-like experience or a development setup with live reloading.

Option A — Single Server (Recommended)

This is the simplest way to run everything. From the `e_banking` folder, just run:

```
python app.py
```

The launcher will automatically build the frontend if it hasn't been built yet, then start Flask on `https://localhost:5001` using a self-signed TLS certificate. Open your browser and go to `https://localhost:5001`.

Note: Your browser will likely show a security warning about the certificate — this is expected. On Windows, you can silence it permanently by running: `certutil -addstore -user Root nginx\ssl\selfsigned.crt`

Option B — Separate Dev Servers (Hot Reload)

If you're actively working on the code, this setup gives you live reloading on the frontend whenever you save a file.

In Terminal 1, start the backend:

```
set ENABLE_TLS=0
python backend/app.py
```

In Terminal 2, start the frontend:

```
cd frontend
npm run dev
```

The backend runs on `http://localhost:5001` and the frontend on `http://localhost:5173`. The frontend is already configured to send API calls to the Flask server, so everything connects automatically.

Step 6 — Verify Everything Is Working

Once the server is running, you can do a quick sanity check by hitting the health endpoint:

```
curl -k https://localhost:5001/health
```

If all is well, you should get back a response like this:

```
{ "message": "E-Banking API is running", "status": "ok" }
```

Step 7 — Running the Tests

The project comes with three test suites covering cryptography, transaction flows, and security. Navigate into the Testing folder and run whichever tests you need:

Unit tests for the crypto layer:

```
python -m pytest Test1_Unit_Crypto/test_crypto.py -v
```

Integration test for the full transaction flow:

```
python Test2_Integration_Transaction/test_transaction_flow.py
```

Security test for replay attack prevention:

```
python Test3_Security_Replay/test_replay_attack.py
```

Troubleshooting

If something goes wrong, here are the most common issues and how to resolve them:

Problem	How to Fix It
<i>pip install fails</i>	Upgrade pip first by running: <code>python -m pip install --upgrade pip</code>
<i>Supabase connection error</i>	Double-check SUPABASE_URL and SUPABASE_KEY values in the .env.backend file
<i>Frontend shows a blank page</i>	Run <code>npm run build</code> inside the frontend folder, then restart Flask
<i>Port 5001 is already in use</i>	The launcher will automatically try port 5002, 5003, and so on — check the console output
<i>Browser shows a certificate warning</i>	Either trust the self-signed certificate (see Step 5) or disable TLS by setting <code>ENABLE_TLS=0</code>
<i>Transaction history returns a 500 error</i>	This is a known bug in GET /transactions/<user> — see Testing/TEST_REPORT.md for details